RETAILMIND AI — ENTERPRISE SPECIFICATION

Company Brain for Smart Retail

Complete A–Z Technical Specification, System Architecture, API Documentation & Operations Manual

Project Name:	RetailMind AI – Smart Retail Operations Suite
Web Frontend:	React 19, Vite, Tailwind CSS v4, Framer Motion, Recharts
Mobile Client:	Flutter SDK ^3.9.2 (Dart), Provider, REST & Snapshot Listeners
Backend Server:	Python FastAPI, Uvicorn, Pydantic v2, ReportLab, QRCode
AI Subsystem:	OpenAI GPT-4o-mini (JSON Order Parser & BI Assistant) + Local Regex Fallback
Data Layer:	Google Cloud Firestore (Live NoSQL) & In-Memory Fallback DB (mock_db.py)
Document Scope:	34 Comprehensive Sections (Architecture, APIs, Schemas, Viva Prep, Troubleshooting)
Generated On:	August 14, 2026 - 11:22 AM

Executive Architectural Summary:
RetailMind AI is an automated, omnichannel retail operations platform designed to bridge customer ordering channels (WhatsApp, Instagram, Web, Email, and Flutter mobile apps) with central inventory management, automated tax invoice generation, and natural-language Business Intelligence (BI) analytics. This document provides a complete technical explanation covering every file, function, API route, database schema, and operational workflow in the repository.

1. PROJECT OVERVIEW & ARCHITECTURE

Main Purpose: Automate the retail ordering pipeline from multi-channel message ingestion through stock verification, tax calculation, and ReportLab PDF invoice generation.

Target Users: Store Administrators (via React Web Dashboard) and Retail Customers (via Flutter Mobile App & Simulated Social Channels).

Dual-Mode Operation: Automatically detects cloud credentials (Firebase/OpenAI). If absent, runs locally using in-memory mock storage and regex heuristics without crashing.

2. COMPLETE TECHNOLOGY STACK

Layer	Technology	Version	Purpose & Role	Source Location
Frontend	React.js + Vite	^19.2 / ^8.1	Single-Page Admin Dashboard	web/src/
Styling	Tailwind CSS v4	^4.3.3	Apple Vibrancy theme, glassmorphism	web/src/index.css
Animations	Framer Motion	^12.42.2	Page transitions & micro-interactions	web/src/components/
Charts	Recharts	^3.9.2	Revenue AreaChart & Category PieChart	web/src/pages/Dashboard
Backend	FastAPI (Python)	>=0.110.0	Asynchronous REST API Gateway	backend/main.py
Server	Uvicorn ASGI	>=0.28.0	ASGI production web server	backend/main.py
Mobile App	Flutter & Dart	SDK ^3.9.2	Cross-platform mobile customer client	mobile/lib/
Database	Cloud Firestore	SDK 6.5 / 12.1	Live NoSQL document synchronisation	Firebase Console / Admin
AI / NLP	OpenAI GPT-4o-mini	>=1.14.0	Structured JSON extraction & BI Chat	backend/services/ai_service
PDF & QR	ReportLab + QRCode	>=4.1 / >=7.4	Dynamic PDF invoice & UPI QR generator	backend/services/pdf_service

3. DIRECTORY & FOLDER STRUCTURE

```
project_root/
■■■ backend/ # Python FastAPI REST API & AI Server
■ ■■■ main.py # Entrypoint, Pydantic models, and 14+ REST endpoints
■ ■■■ config.py # Environment loader & mock decision engine
■ ■■■ mock_db.py # In-memory database tables and activity loggers
■ ■■■ services/
■ ■■■ ai_service.py # OpenAI GPT-4o-mini & rule-based regex fallback
■ ■■■ firestore_listener.py # Async daemon thread for order AI analysis
■ ■■■ pdf_service.py # ReportLab PDF invoice generator with UPI QR codes
■■■ web/ # React 19 Vite Admin Dashboard
■ ■■■ src/
■ ■■■ App.jsx # Router, page layout, and command palette
■ ■■■ config.js # Dynamic API URL resolver (localStorage + env)
■ ■■■ firebase.js # Firebase Web client SDK initialization
■ ■■■ components/ # Sidebar, CommandPalette, NotificationSystem, QuickActions
■ ■■■ pages/ # 11 pages (Dashboard, Orders, Inventory, Products, etc.)
■■■ mobile/ # Flutter Customer Mobile Application
■■■ lib/
■■■ main.dart # Entrypoint, dark theme, Provider injection
■■■ services/ # firebase_service.dart (Backend URL detector & API client)
■■■ screens/ # 7 screens (Login, Register, Home, OrderForm, History, etc.)
```

4. FRONTEND (REACT WEB ADMIN DASHBOARD) — 11 PAGES

- **Dashboard (/):** Executive KPI overview (Revenue, Pending, Low Stock, SKUs), Recharts AreaChart & PieChart, live synced recent orders table. (web/src/pages/Dashboard.jsx)
- **Orders (/orders):** Live triage center. Displays platform pill tags (WhatsApp, IG, Web, Email), AI product matching results, stock shortage alerts, substitute suggestions, and 1-click status transitions. (web/src/pages/Orders.jsx)
- **Inventory (/inventory):** Stock level tracking by SKU. Interactive +/- stepper adjustments, critical threshold alerts (<10 units), and real-time database writeback. (web/src/pages/Inventory.jsx)
- **Products (/products):** Catalog management suite. Add/Edit/Delete products, image previews, category tags (Shoes, Apparel, Electronics, Accessories), and price range filtering. (web/src/pages/Products.jsx)
- **Customers (/customers):** Customer CRM directory. Tracks contact metadata, aggregate lifetime spending, and total order counts. (web/src/pages/Customers.jsx)
- **Invoices (/invoices):** Financial ledger of all completed/processing orders. Calculates 18% GST breakdowns and provides 1-click direct downloads of ReportLab PDF invoices. (web/src/pages/Invoices.jsx)
- **AI Assistant (/ai-assistant):** Conversational BI interface. Connects to current store inventory and sales context using OpenAI GPT-4o-mini with fallback heuristic answers. (web/src/pages/AIAssistant.jsx)
- **Reports (/reports):** Operations overview with stock units by category and 1-click export of sales records to formatted CSV. (web/src/pages/Reports.jsx)
- **Analytics (/analytics):** Deep temporal revenue analysis, day-of-week sales frequency, hourly distribution, and top product performers. (web/src/pages/Analytics.jsx)
- **Channels (/channels):** Omnichannel manager and simulator for WhatsApp, Instagram DM AI, Website Store, and Email webhooks. (web/src/pages/Channels.jsx)
- **Settings (/settings):** Store branding config, GST tax rate parameters, and dynamic backend API base URL switching with live health ping. (web/src/pages/Settings.jsx)

5. BACKEND & API ROUTES DOCUMENTATION

Method	Endpoint	Purpose	Request Body / Params	Response
GET	/api/status	Server health check	None	{ status, mock_db, mock_ai }
GET	/api/products	List & filter products	?category=...&search;=...	List[ProductModel]
POST	/api/products	Create catalog item	ProductModel JSON	Created Product Object
PUT	/api/inventory/{id}	Adjust stock units	{ stock: int }	Updated Product Object
GET	/api/orders	List customer orders	?platform=whatsapp/ig/...	List[OrderModel]
POST	/api/orders	Create new order	OrderCreateModel JSON	Pending Order Object
PUT	/api/orders/{id}/status	Transition status & stock	{ status: 'Processing'/'Completed' }	Updated Order Object
GET	/api/orders/{id}/invoice	Download PDF invoice	Path order_id	FileResponse (PDF Stream)
POST	/api/ai/chat	Query AI BI Assistant	{ question: str }	{ response: str }
GET	/api/reports	Aggregated KPI metrics	None	{ metrics: {...}, platforms: {...} }
GET	/api/reports/export	Download sales CSV	None	StreamingResponse (CSV)
GET	/api/notifications	Fetch system alerts	None	List[Notification]

6. FLUTTER MOBILE CLIENT (7 SCREENS)

- **LoginScreen (login_screen.dart):** Animated ambient mesh login with email/password validation and 1-tap demo credentials.
- **RegisterScreen (register_screen.dart):** Customer sign-up creating Firebase Auth user and Firestore user profile.
- **HomeScreen / CustomerDashboardView (home_screen.dart):** Customer landing page with live order metrics, omnichannel action launchers, and catalog highlights.
- **OrderFormScreen (order_form_screen.dart):** Dual-mode order screen: Tab 1 for structured cart shopping; Tab 2 for natural language NLP smart text messaging.
- **OrderHistoryScreen (order_history_screen.dart):** Real-time reactive order tracker with 4-step status timeline (Order Placed → AI Analyzed → Processing → Completed) and PDF download button.
- **NotificationsScreen (notifications_screen.dart):** Alert stream for order updates, low-stock warnings, and invoice generation.
- **ProfileScreen (profile_screen.dart):** Customer address preferences, phone number management, and sign-out controls.

7. AI & MACHINE LEARNING SUBSYSTEM

1. Unstructured Order Extraction: Powered by OpenAI gpt-4o-mini in JSON mode (temperature 0.0). Accepts customer text and extracts: products: [{ name, quantity }], customer name, phone, address, and delivery instructions. If offline, runs a local regex parser with stop-word filtering.

2. Autonomous Worker Daemon: Spawns in a background thread on server startup (firestore_listener.py). Detects unprocessed orders, matches catalog items, evaluates stock availability, generates substitute suggestions if out-of-stock, calculates 18% GST, and dispatches in-app notifications.

3. Conversational BI Assistant: Connects to active inventory and recent orders context to answer executive questions like daily revenue and low-stock alerts.

8. REPORTLAB PDF INVOICE & QR ENGINE

The invoice compiler in backend/services/pdf_service.py builds a SimpleDocTemplate containing:

- **Branded Two-Column Header:** RetailMind AI logo on left; Invoice #INV-xxxx, Date, Platform, and Status on right.
- **Bill From / Bill To Box:** Company HQ address, GSTIN (22AAAAA0000A1Z5), customer name, phone, and delivery destination.
- **Itemized Pricing Table:** Line items with Unit Prices, Quantities, and Line Totals in INR.
- **Tax Calculations:** Subtotal, GST (18%), and Grand Total.
- **Embedded 2D QR Code:** Generated in memory using Python qrcode + Pillow containing invoice verification data and UPI gateway links.

9. DATABASE SCHEMA (FIRESTORE & MOCK DB)

Collection / Table	Key Fields	Data Types	Purpose & Relationship
products	productId (PK), name, category, price, stock, sku, supplier, image	str, str, str, float, int, str, str, str	Store inventory catalog. 1-to-many with inventory_logs.
orders	orderId (PK), customerId (FK), customerName, phone, address, platform, message, products, subtotal, gst, total, status, aiProcessed, aiSuggestions, timestamp	str, str, str, str, str, str, str, list[dict], float, float, float, str, bool, list[str], float	Omnichannel order records. Many-to-1 with customers, 1-to-1 with invoices.
customers	uid (PK), name, email, phone, address, totalPurchases, previousOrders	str, str, str, str, str, float, int	Customer CRM profiles. 1-to-many with orders.
invoices	invoiceId (PK), orderId (FK), customerName, total, pdfUrl, timestamp	str, str, str, float, str, timestamp	Compiled invoice records with PDF download routes.
notifications	notificationId (PK), title, message, type, orderId (FK), read, timestamp	str, str, str, str, str, bool, timestamp	Real-time alerts for new orders, stockouts, and invoices.
inventory_logs	logId (PK), productId (FK), productName, previousStock, newStock, reason, timestamp	str, str, str, int, int, str, timestamp	Audit trail for stock adjustments and order deductions.

10. TOP 5 INTERVIEW & VIVA QUESTIONS

Q1: What problem does RetailMind AI solve?

It automates the omnichannel retail pipeline. It takes messy, unstructured messages from WhatsApp/Instagram, uses OpenAI GPT-4o-mini & regex to extract catalog products, verifies real-time stock, calculates 18% GST, raising restock alerts, and generates ReportLab PDF invoices with QR codes.

Q2: How does the system work without OpenAI or Firebase API keys?

It features an intelligent Dual-Mode Architecture. In config.py, the backend detects missing credentials and activates in-memory storage (mock_db.py), local regex extraction (ai_service.py), and polling streams, allowing 100% offline evaluation out-of-the-box.

Q3: How does Flutter communicate with FastAPI on an Android Emulator?

In firebase_service.dart, detectBackend() probes candidate IPs including http://10.0.2.2:8000 (which maps to host 127.0.0.1:8000) and dynamically selects the active route.

Q4: How does stock deduction ensure inventory integrity?

When an order status is updated to Processing or Completed in main.py, the backend iterates over line items, reduces available stock in products, records an audit record in inventory_logs, and triggers a low-stock alert if units drop below 10.

Q5: How are PDF invoices generated?

Using Python's ReportLab library in pdf_service.py. It constructs a SimpleDocTemplate with branded typography, itemized tables, GST calculations, and an embedded 2D QR code generated via qrcode + Pillow.

11. 5-MINUTE VIVA / PRESENTATION PITCH

"RetailMind AI is an omnichannel smart retail operations suite synchronising a React web dashboard, a customer Flutter mobile app, and a Python FastAPI backend.

When customers message on WhatsApp or Instagram, our AI engine (GPT-4o-mini with regex fallbacks) extracts items, quantities, and addresses. A background daemon matches items against active inventory, flags stock shortages with AI substitute suggestions, and adds 18% GST. When the store manager approves the order on the React dashboard, inventory decrements automatically and a ReportLab PDF invoice with a UPI QR code is compiled instantly. Store owners can also query our AI BI assistant in plain English to analyze daily sales and restock needs."

12. FINAL A–Z CHEAT SHEET SUMMARY

- **A** — Architecture (React + Flutter + FastAPI + Firestore/Mock DB)
- **B** — Backend (FastAPI ASGI server on port 8000)
- **C** — Channels (WhatsApp, Instagram, Web, Email)
- **D** — Data Flow (Raw Message → AI Parse → Stock Check → GST 18% → PDF Invoice)
- **E** — Endpoints (14+ REST API endpoints)
- **F** — Frontend (React 19 + Vite + Tailwind CSS v4 + Framer Motion)
- **G** — GST Calculation (Automated 18% tax addition)
- **H** — Heuristic Fallback (Regex parser when OpenAI is offline)
- **I** — Invoices (ReportLab programmatic PDF + 2D QR Code)
- **M** — Mobile (Flutter client with structured cart & NLP ordering)
- **O** — OpenAI (GPT-4o-mini structured JSON & BI assistant)
- **P** — Products (Full catalog CRUD & SKU tracking)
- **Q** — QR Codes (Generated via qrcode + Pillow in PDF invoices)
- **S** — Stock Control (Auto-deduction on fulfillment + critical alerts)
- **Z** — Zero Cloud Dependency (100% functional offline mock mode)